

Foodeez Flutter App

Workflow, State Management, and Project Structure Documentation

Role-specific mobile application for **restaurant_admin**

Generated for project: **foodeez_flutter**

1. Overview

This document explains the Flutter mobile app architecture, navigation workflow, Riverpod state management flow, API handling, data model parsing, and screen/module responsibilities. The application is designed only for users with the role **restaurant_admin**.

2. High-Level Architecture

The app follows an MVC-like layered structure. Views render UI, Providers manage state and connect UI to services, Services call backend APIs, and Models safely convert API responses into Dart objects.

View → Provider → Service → API → Model → Provider State → View

Layer Responsibilities

Layer	Responsibility
Views	Flutter screens and widgets. They watch Riverpod providers and render loading, error, or data states.
Providers	Riverpod state layer. Handles auth state, API FutureProviders, filters, controls, and live order state.
Services	API integration layer. Uses Dio to call backend endpoints and returns parsed models.
Models	Data classes. Convert JSON to strongly typed Dart objects using safe parsing.
Core	Shared constants, theme, Dio client, storage, JWT utilities, crypto utilities, and formatters.

3. Project Folder Structure

```
lib/
  main.dart
  app.dart
  router/
    app_router.dart
  core/
    constants/
    network/
    storage/
    theme/
    utils/
  models/
  services/
  providers/
  views/
    auth/
    splash/
    dashboard/
    branches/
    orders/
    kds/
```

```
■ ■■■ settlement/  
■ ■■■ documents/  
■■■ widgets/
```

4. App Startup Flow

```
main.dart  
↓  
ProviderScope  
↓  
FoodeezApp  
↓  
GoRouter  
↓  
SplashScreen  
↓  
Auth state check  
↓  
/dashboard if valid restaurant_admin  
/login if no token, expired token, or wrong role
```

The splash screen is the first route. It waits briefly, checks the authentication state, and sends the user either to the dashboard or login screen.

5. Authentication Flow

Login Process

```
LoginScreen  
↓ user enters email/password  
AuthNotifier.login(email, password)  
↓  
AuthService.login()  
↓  
POST /auth/login  
↓  
accessToken returned  
↓  
Decode JWT  
↓  
Check role == restaurant_admin  
↓  
Save token in secure storage  
↓  
AuthState updated  
↓  
Router redirects to /dashboard
```

Role Restriction

The app is restricted to only this role:

```
{ "role": "restaurant_admin" }
```

If the user logs in with any other role such as super admin, sales operator, delivery partner, or customer, the token is not saved and access is denied.

```
if (role != AppConstants.roleRestaurantAdmin) {  
  throw ApiException(  
    'Access denied. This app is only for Restaurant Admins.',  
    statusCode: 403,  
  );  
}
```

```
);  
}
```

Stored Token Validation

On app restart, stored tokens are decoded again. If the token is expired or if the role is not `restaurant_admin`, the token is cleared and the user is redirected to login.

6. Riverpod State Management

The app uses Riverpod for dependency injection, asynchronous API state, and mutable application state.

Provider Types

Provider Type	Used For	Example
Provider	Dependency injection for service classes	<code>authServiceProvider</code> , <code>branchServiceProvider</code>
FutureProvider	API data fetching with loading/error/data state	<code>rawDashboardProvider</code> , <code>settlementOrdersProvider</code>
FutureProvider.family	API calls requiring parameters	<code>branchesProvider(restaurantId)</code>
NotifierProvider	Mutable state and actions	<code>authProvider</code> , <code>liveOrdersProvider</code>

Typical Data Flow

```
Screen watches provider  
↓  
Provider calls service  
↓  
Service calls API with Dio  
↓  
Response parsed into model  
↓  
Provider returns AsyncValue  
↓  
Screen renders loading/error/data
```

AsyncValue UI Pattern

```
final branchesAsync = ref.watch(branchesProvider(restaurantId));  
  
branchesAsync.when(  
  loading: () => const FullPageLoader(),  
  error: (e, _) => ErrorView(message: e.toString()),  
  data: (branches) => ListView(...),  
);
```

7. Navigation Flow

Navigation is centralized in Go Router. The router protects private screens and redirects unauthenticated users to login.

Main Routes

```
/splash
/login
/dashboard
/restaurant
/branches
/branches/new
/branches/:branchId
/branches/:branchId/menu
/orders
/kds
/settlement
/documents
/users
```

Back Navigation Rules

- Dashboard shows hamburger menu and app logo.
- Other top-level pages show a back arrow that navigates to /dashboard.
- Deep sub-screens use context.pop() to return to the previous page.

8. API Layer

All backend calls are made through Dio. The Dio client attaches the stored token automatically and converts API errors into ApiException.

```
Screen
↓
Provider
↓
Service
↓
Dio Client
↓
Backend API
↓
Model parsing
↓
UI update
```

API Client Responsibilities

- Set base API URL from environment constants.
- Attach Authorization bearer token when available.
- Log useful debug request details in development.
- Convert DioException into readable ApiException messages.

9. Model Parsing Strategy

The backend can return numeric values as strings. To avoid runtime crashes, models use safe parsing helpers instead of direct casts like value as num?.

```
double _toDouble(dynamic value) {
  if (value == null) return 0;
  if (value is num) return value.toDouble();
  return double.tryParse(value.toString()) ?? 0;
}
```

This prevents errors such as: type 'String' is not a subtype of type 'num?' in type cast.

10. Module Workflows

Dashboard

```
DashboardScreen
↓
rawDashboardProvider
↓
RestaurantService.getRawDashboard()
↓
GET /dashboard
↓
Render total branches, branch status, live orders, onboarding, and branch list
```

Branches

```
Dashboard → Manage Branches
↓
/branches
↓
branchesProvider(restaurantId)
↓
BranchService.getBranches()
↓
GET /restaurants/:restaurantId/branches
↓
List<BranchModel>
↓
BranchesScreen renders branch cards
```

Branch actions include menu navigation, controls navigation, and online/offline toggle.

Orders

```
/orders
↓
ordersProvider
↓
OrderService.getOrders()
↓
GET /restaurant/orders
↓
List<OrderModel>
↓
OrdersScreen renders status-filtered orders
```

KDS

```
/kds
↓
Connect socket
↓
Join restaurant room
↓
Listen for new-order events
```

```
↓  
Poll active orders periodically  
↓  
Accept / Reject / Mark Ready actions update order state
```

Settlement

```
SettlementScreen  
↓  
SettlementService.getTodayOrders()  
↓  
GET /restaurant/orders?status=DELIVERED  
↓  
Calculate gross revenue, 5% platform fee, and net payout
```

11. Environment Handling

API URLs and build-time configuration are managed with Dart defines. This allows local and production API URLs without code changes.

```
flutter run --dart-define=API_BASE_URL=http://localhost:3000/api/v1  
  
flutter build apk --dart-define=API_BASE_URL=https://your-production-api.com/api/v1
```

12. Summary

The Foodeez Flutter app is structured to keep UI, state, API, and data conversion separate. Riverpod manages both async API state and mutable app state. Go Router controls navigation and auth protection. Services isolate backend integration. Models safely parse backend data, including string numbers, to prevent runtime type errors.

```
main.dart  
↓  
ProviderScope  
↓  
FoodeezApp  
↓  
GoRouter  
↓  
Splash/Login/Dashboard  
↓  
Screens watch Riverpod providers  
↓  
Providers call Services  
↓  
Services call APIs  
↓  
Models parse response safely  
↓  
UI updates automatically
```